RAPIDS

The Platform Inside and Out

@RAPIDSai

https://github.com/rapidsai

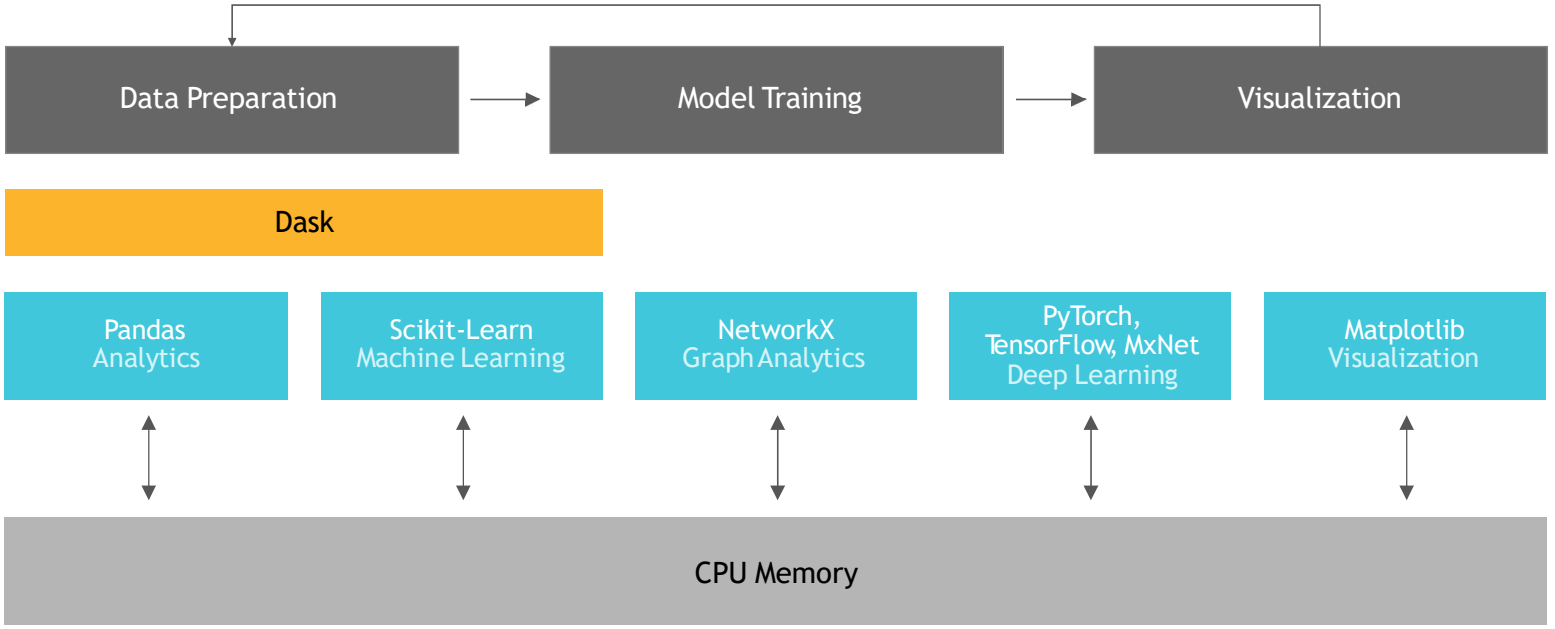
https://rapids-goai.slack.com/join

RAPIDS

https://rapids.ai

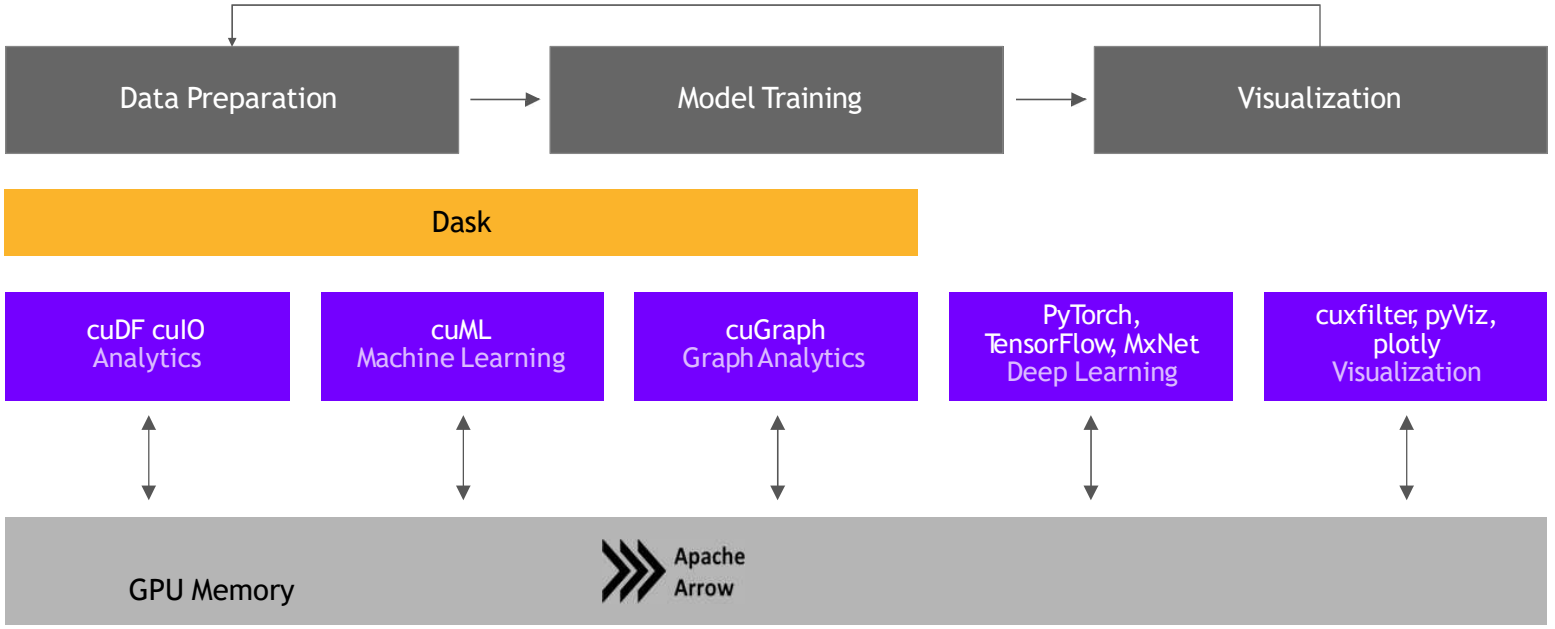
Open Source Data Science Ecosystem

Familiar Python APIs



RAPIDS

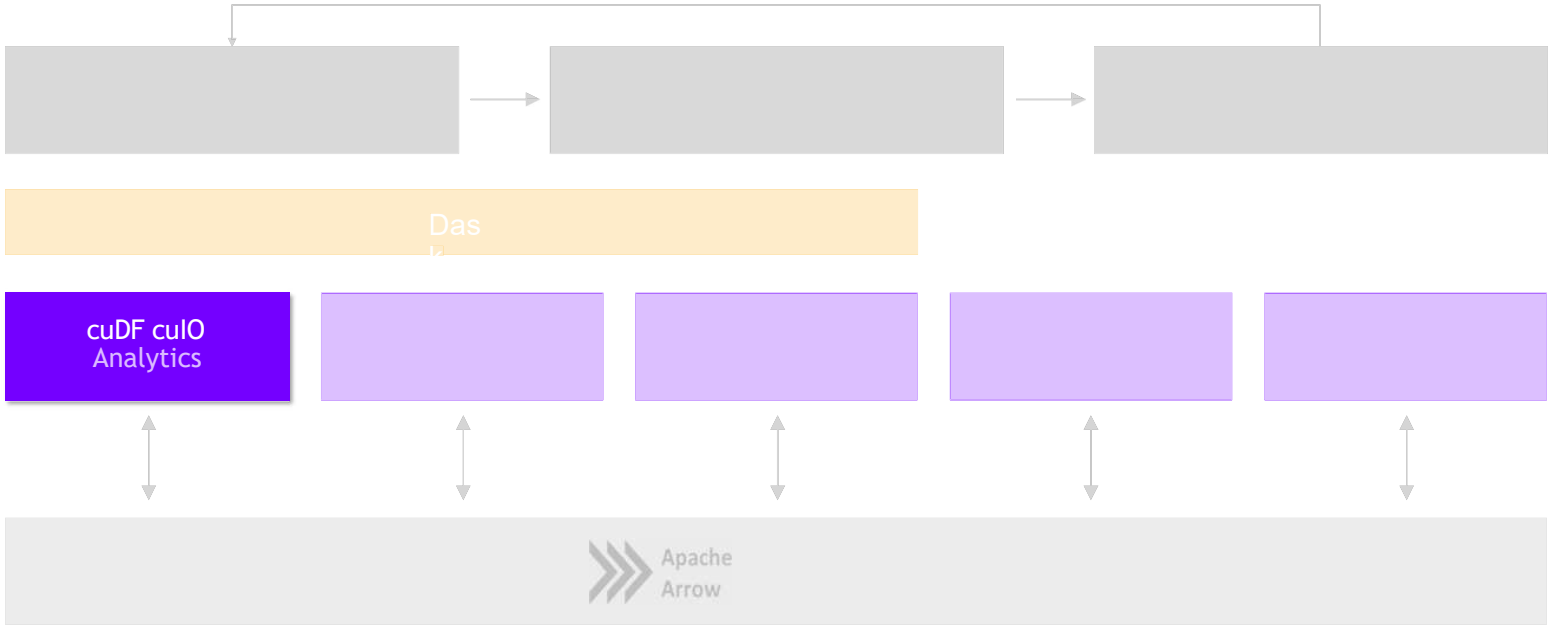
End-to-End Accelerated GPU Data Science



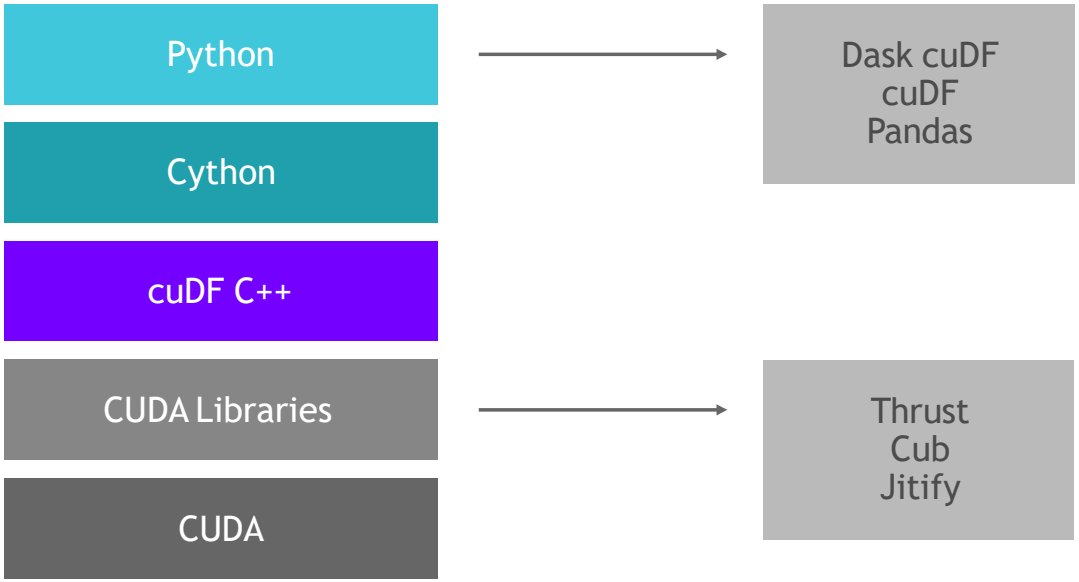
cuDF

RAPIDS

GPU Accelerated Data Wrangling and Feature Engineering



ETL Technology Stack



ETL - the Backbone of Data Science

libcuDF is…

CUDA C++ LIBRARY

- Table (dataframe) and column types and algorithms
- CUDA kernels for sorting, join, groupby, reductions, partitioning, elementwise operations, etc.
- Optimized GPU implementations for strings, timestamps, numeric types (more coming)
- Primitives for scalable distributed ETL

```
std::unique_ptr<table>
gather(table_view const& input,
       column_view const& gather_map, …)
{
        // return a new table containing
    // rows from input indexed by
    // gather_map
}
```


ETL - the Backbone of Data Science

cuDF is…

PYTHON LIBRARY

- A Python library for manipulating GPU DataFrames following the Pandas API
- Python interface to CUDA C++ library with additional functionality
- Creating GPU DataFrames from Numpy arrays, Pandas DataFrames, and PyArrow Tables
- JIT compilation of User-Defined Functions (UDFs) using Numba

```
In [2]: #Read in the data. Notice how it decompresses as it reads the data into memory.
gdf = cudf.read_csv('/rapids/Data/black-friday.zip')
```

```
In [3]: #Taking a look at the data. We use "to_pandas()" to get the pretty printing.
gdf.head().to_pandas()
```

Out[3]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Ca
0	1000001	P00069042	F	0-17	10	A	2	0	3
1	1000001	P00248942	F	0-17	10	A	2	0	1
2	1000001	P00087842	F	0-17	10	A	2	0	12
3	1000001	P00085442	F	0-17	10	A	2	0	12
4	1000002	P00285442	M	55+	16	C	4+	0	8

```
In [6]: #grabbing the first character of the years in city string to get rid of plus sign, and converting
 to int
gdf['city_years'] = gdf.Stay_In_Current_City_Years.str.get(0).stoi()
```

```
In [7]: #Here we can see how we can control what the value of our dummies with the replace method and turn
strings to ints
gdf['City_Category'] = gdf.City_Category.str.replace('A', '1')
gdf['City_Category'] = gdf.City_Category.str.replace('B', '2')
gdf['City_Category'] = gdf.City_Category.str.replace('C', '3')
gdf['City_Category'] = gdf['City_Category'].str.stoi()
```

ETL - the Backbone of Data Science

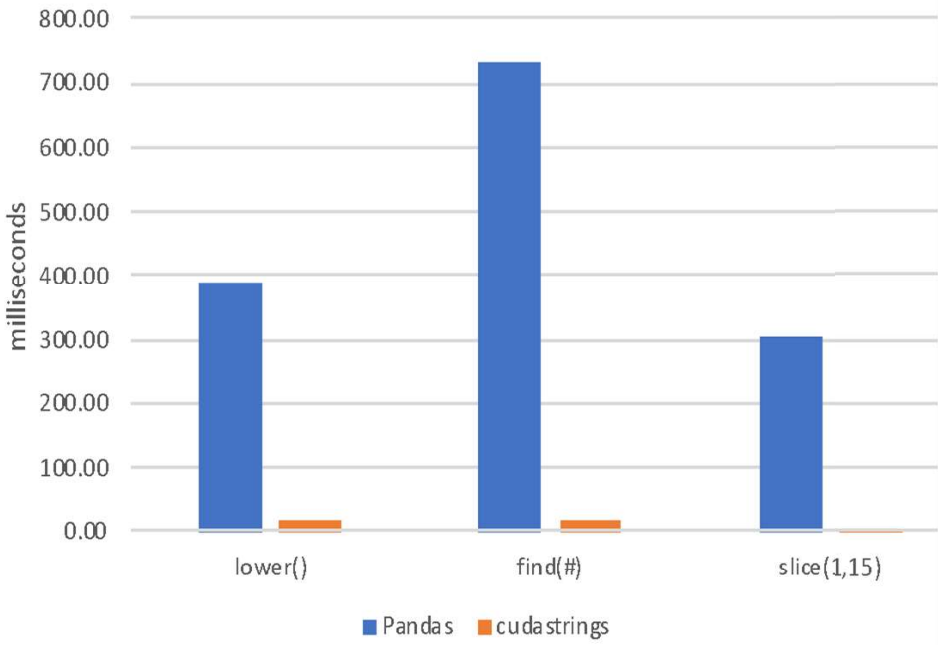
String Support

Current v0.9 String Support

- Regular Expressions
- Element-wise operations
  - Split, Find, Extract, Cat, Typecasting, etc…
- String GroupBys, Joins
- Categorical columns fully on GPU

Future v0.10+ String Support

- Combining cuStrings into libcudf
- Extensive performance optimization
- More Pandas String API compatibility
- JIT-compiled String UDFs



Extraction is the Cornerstone

cuIO for Faster Data Loading

- Follow Pandas APIs and provide >10x speedup
- CSV Reader - v0.2, CSV Writer v0.8
- Parquet Reader - v0.7, Parquet Writer v0.10
- ORC Reader - v0.7, ORC Writer v0.10
- JSON Reader - v0.8
- Avro Reader - v0.9
- GPU Direct Storage integration in progress for bypassing PCIe bottlenecks!
- Key is GPU-accelerating both parsing and decompression wherever possible

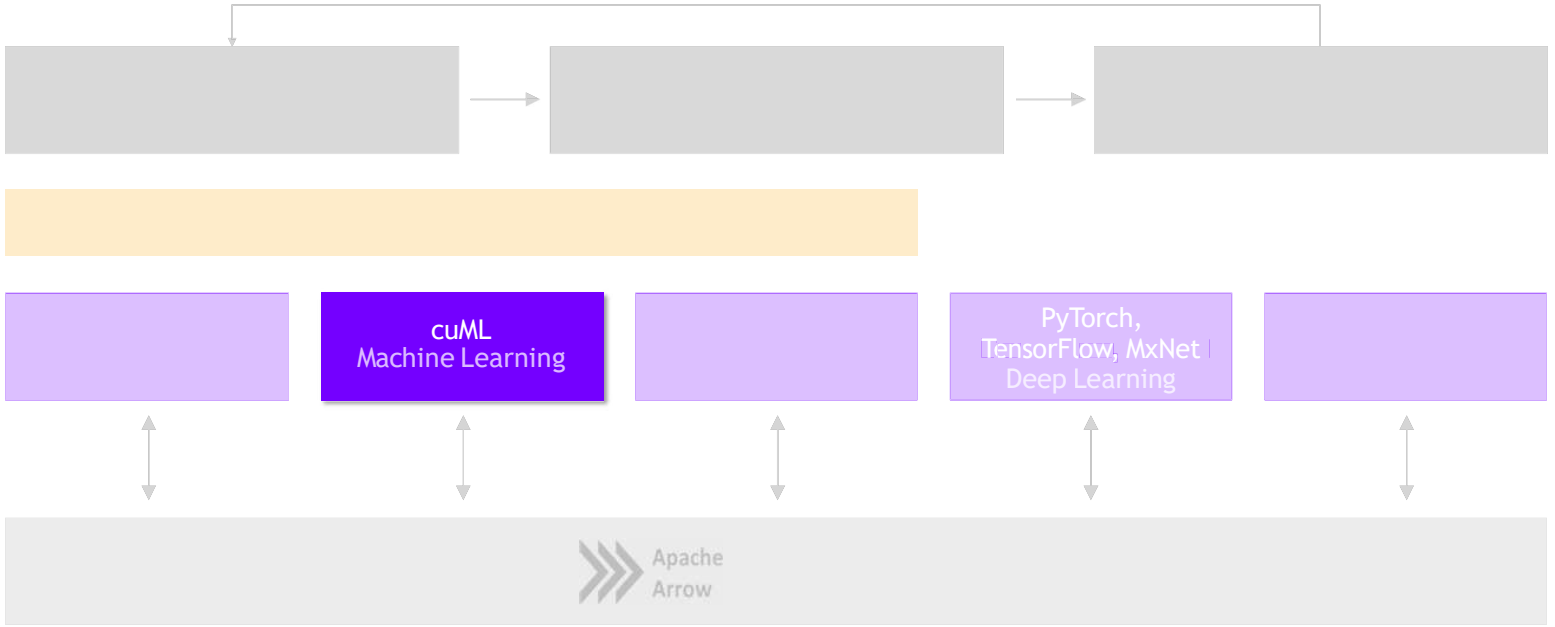
```
1]: import pandas, cudf

2]: %time len(pandas.read_csv('data/nyc/yellow_tripdata_2015-01.csv'))
CPU times: user 25.9 s, sys: 3.26 s, total: 29.2 s
Wall time: 29.2 s
2]: 12748986

3]: %time len(cudf.read_csv('data/nyc/yellow_tripdata_2015-01.csv'))
CPU times: user 1.59 s, sys: 372 ms, total: 1.96 s
Wall time: 2.12 s
3]: 12748986

4]: !du -hs data/nyc/yellow_tripdata_2015-01.csv
1.9G    data/nyc/yellow_tripdata_2015-01.csv
```

Source: Apache Crail blog: SQL Performance: Part 1 - Input File Formats

cuML

Machine Learning

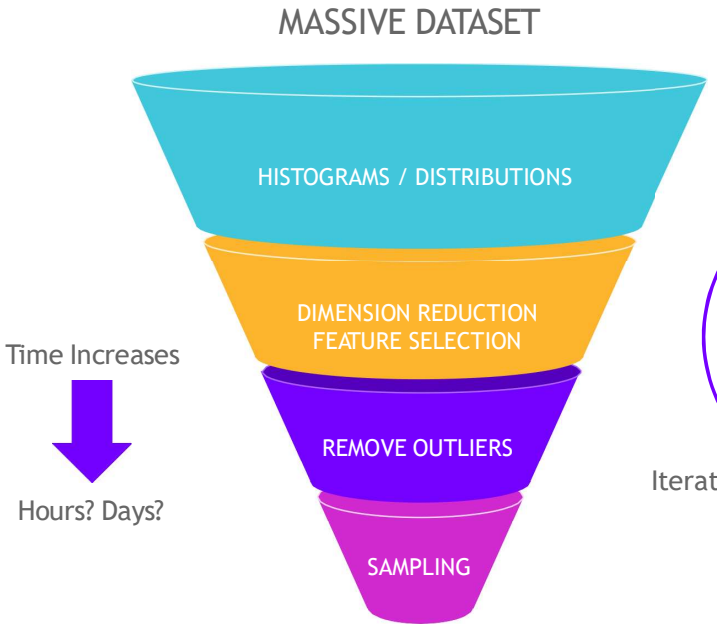
More Models More Problems

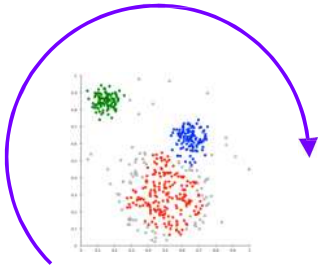


Problem

Data Sizes Continue to Grow

Better to start with as much data as possible and explore / preprocess to scale to performance needs.

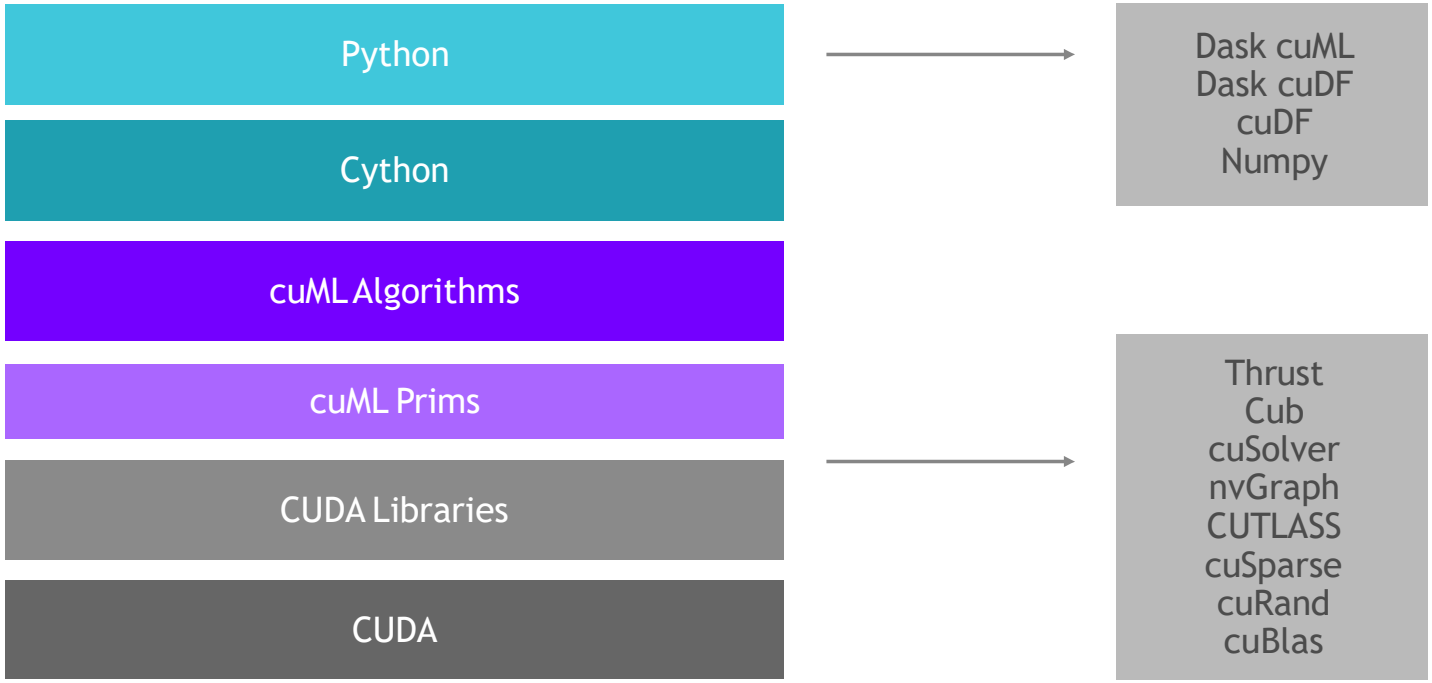




Iterate. Cross Validate & Grid Search.
Iterate Some More.

Meet Reasonable Speed vs Accuracy Trade-off

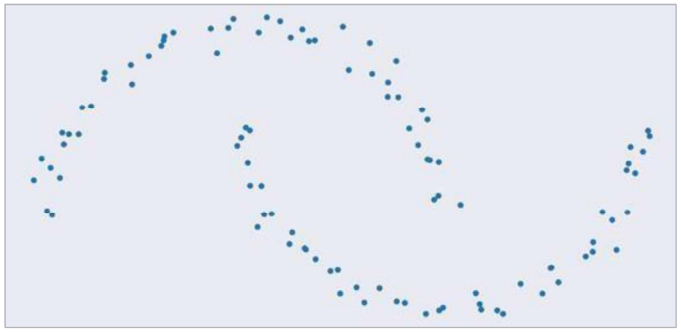
ML Technology Stack



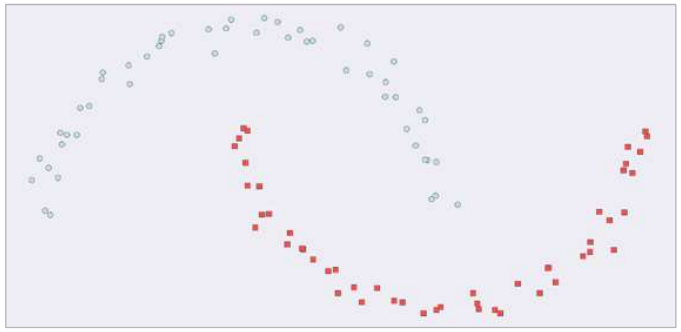
RAPIDS Matches Common Python APIs

CPU-based Clustering

```
from sklearn.datasets import make_moons
import pandas

X, y = make_moons(n_samples=int(1e2),
                  noise=0.05, random_state=0)

X = pandas.DataFrame({'fea%d'%i: X[:, i]
              for i in range(X.shape[1])})
```



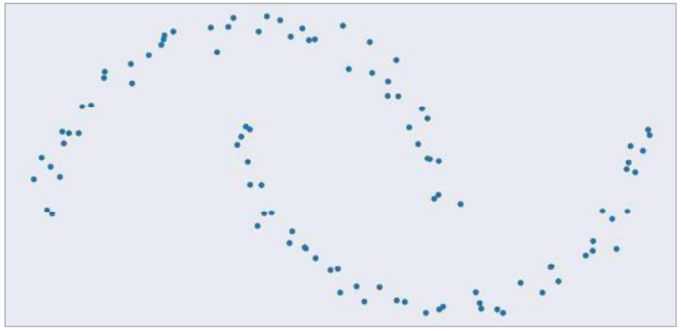
```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps = 0.3, min_samples = 5)

y_hat = dbscan.fit_predict(X)
```



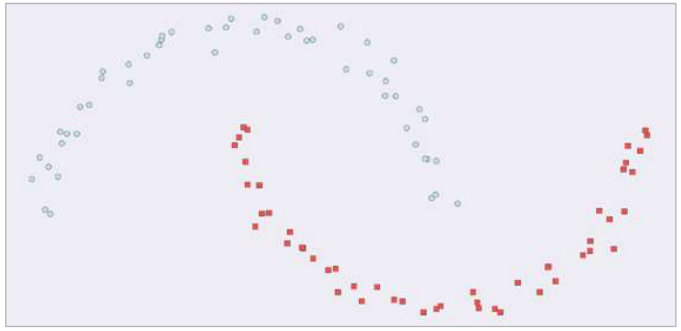
RAPIDS Matches Common Python APIs

GPU-accelerated Clustering

```
from sklearn.datasets import make_moons
import cudf


X, y = make_moons(n_samples=int(1e2),
           noise=0.05, random_state=0)

X = cudf.DataFrame({'fea%d'%i: X[:, i]
           for i in range(X.shape[1])})
```

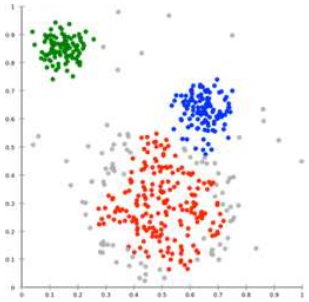


```
from cuml import DBSCAN
dbscan = DBSCAN(eps = 0.3, min_samples = 5)


y_hat = dbscan.fit_predict(X)
```



Algorithms

GPU-accelerated Scikit-Learn



Cross Validation

Hyper-parameter Tuning

More to come!

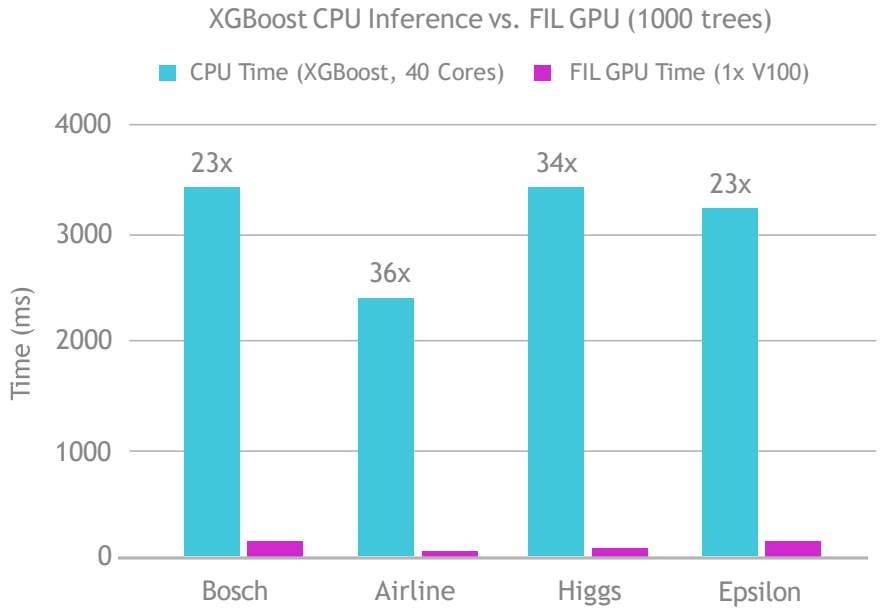
Classification / Regression	Decision Trees / **Random Forests** Linear/Lasso/Ridge/**LARS**/ElasticNet Regression Logistic Regression K-Nearest Neighbors (**exact or approximate**) Support Vector Machine Classification and Regression Naive Bayes
Inference	Random Forest / GBDT Inference (FIL)
Preprocessing	Text vectorization (TF-IDF / Count) Target Encoding Cross-validation / splitting
Clustering Decomposition & Dimensionality Reduction	K-Means **DBSCAN** Spectral Clustering Principal Components (including iPCA) Singular Value Decomposition UMAP Spectral Embedding T-SNE
Time Series	Holt-Winters Seasonal ARIMA / AutoARIMA

Forest Inference

Taking Models From Training to Production

cuML's Forest Inference Library accelerates prediction (inference) for random forests and boosted decision trees:

- Works with existing saved models (XGBoost, LightGBM, scikit-learn RF cuML RF soon)
- Lightweight Python API
- Single V100 GPU can infer up to 34x faster than XGBoost dual-CPU node
- Over 100 million forest inferences



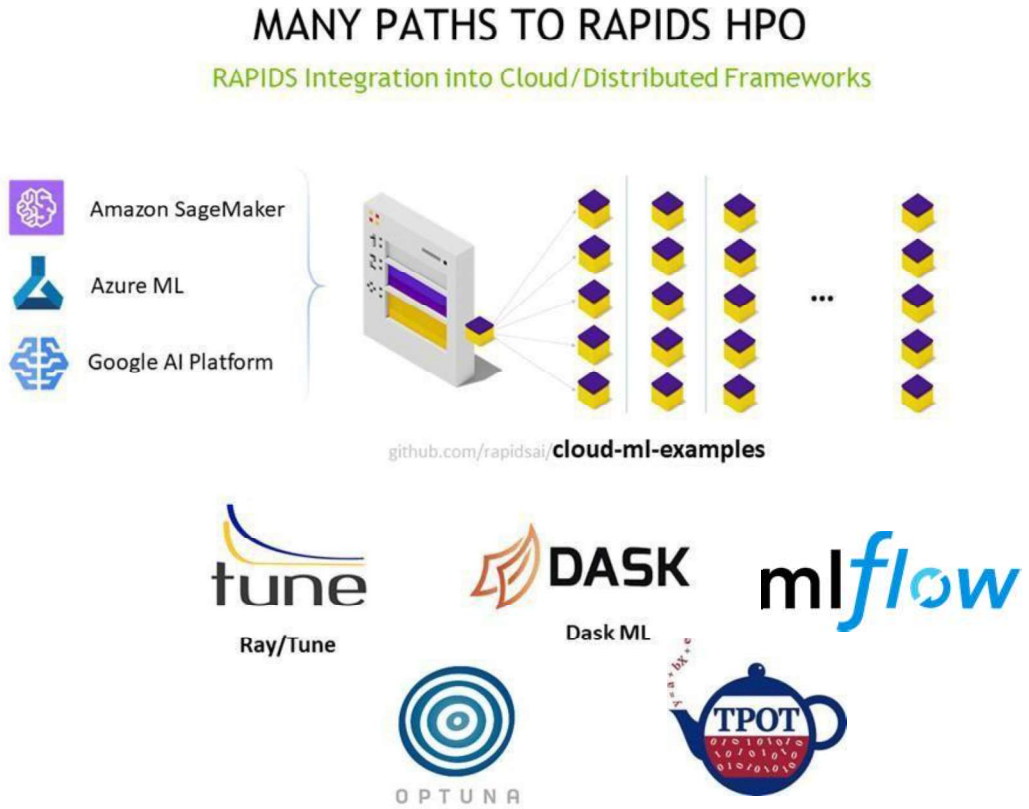
RAPIDS Integrated into Cloud ML Frameworks

Accelerated machine learning models in RAPIDS give you the flexibility to use hyperparameter optimization (HPO) experiments to explore all variants to find the most accurate possible model for your problem.

With GPU acceleration, RAPIDS models can train 25x faster than CPU equivalents, enabling more experimentation in less time.

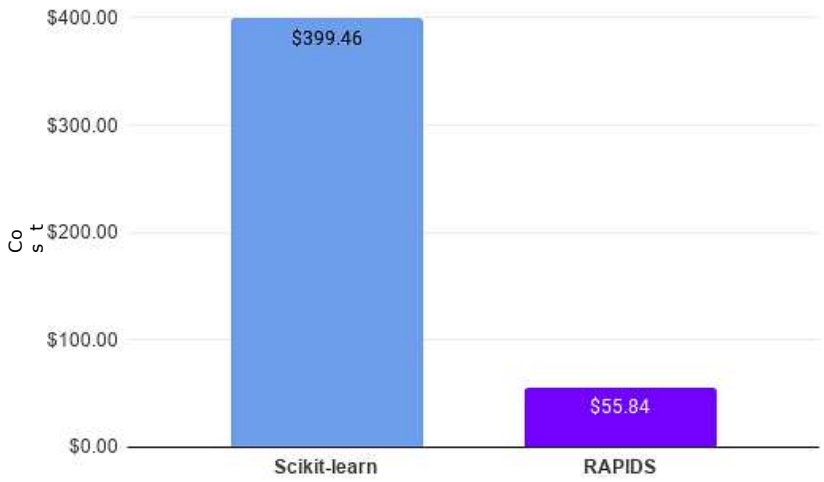
The RAPIDS team works closely with major cloud providers and OSS solution providers to provide code samples to get started with HPO in minutes.

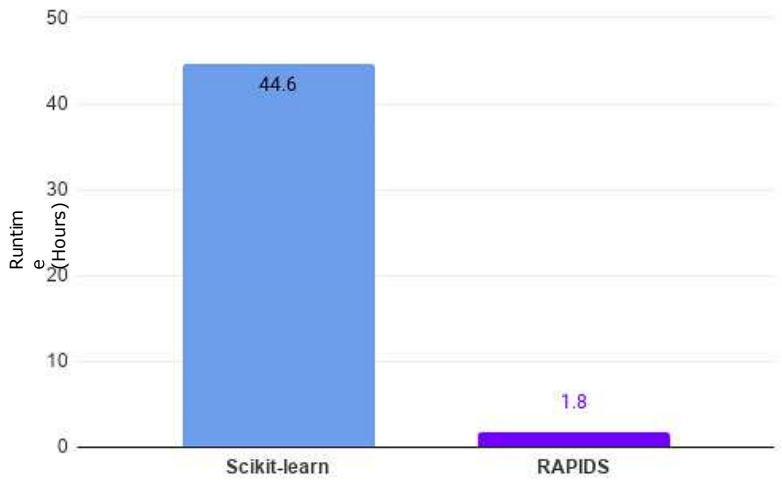
https://rapids.ai/hpo



HPO Use Case: 100-Job Random Forest Airline Model

Huge speedups translate into >7x TCO reduction





Based on sample Random Forest training code from cloud-ml-examples repository, running on Azure ML. 10 concurrent workers with 100 total runs, 100M rows, 5-fold cross-validation per run.

GPU nodes: 10x Standard_NC6s_v3, 1 V100 16G, vCPU 6 memory 112G, Xeon E5-2690 v4 (Broadwell) - $3.366/hour
CPU nodes: 10x Standard_DS5_v2, vCPU 16 memory 56G, Xeon E5-2673 v3 (Haswell) or v4 (Broadwell) - $1.017/hour"

Overview of cuML Algorithms

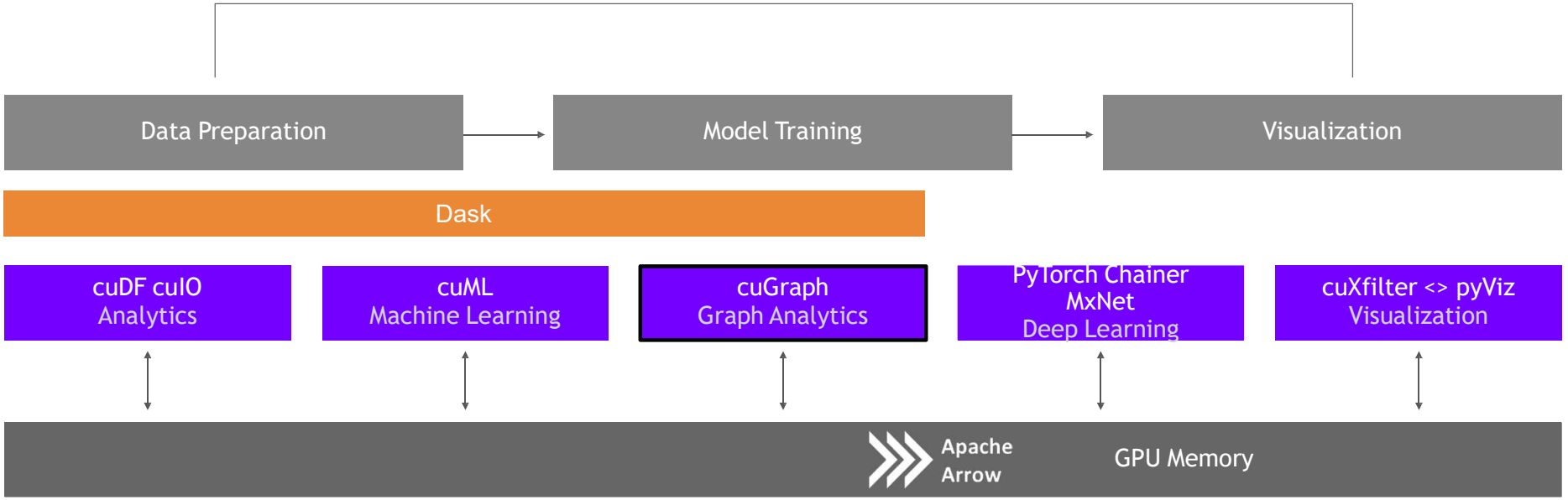
RAPIDS 0.18 - February 2021

cuML	Single-GPU	Multi-Node-Multi-GPU
Gradient Boosted Decision Trees (GBDT)		
Linear Regression		
Logistic Regression		
Random Forest		
K-Means		
K-NN		
DBSCAN		
UMAP		
Holt-Winters		
ARIMA		
T-SNE		
Principal Components		
Singular Value Decomposition		
SVM		

cuGraph

Graph Analytics

More connections more insights



GOALS AND BENEFITS OF CUGRAPH

Focus on Features and User Experience

Breakthrough Performance

- Up to 500 million edges on a single 32GB GPU
- Multi-GPU support for scaling into the billions of edges

Multiple APIs

- **Python**: Familiar NetworkX-like API
- **C/C++:** lower-level granular control for application developers

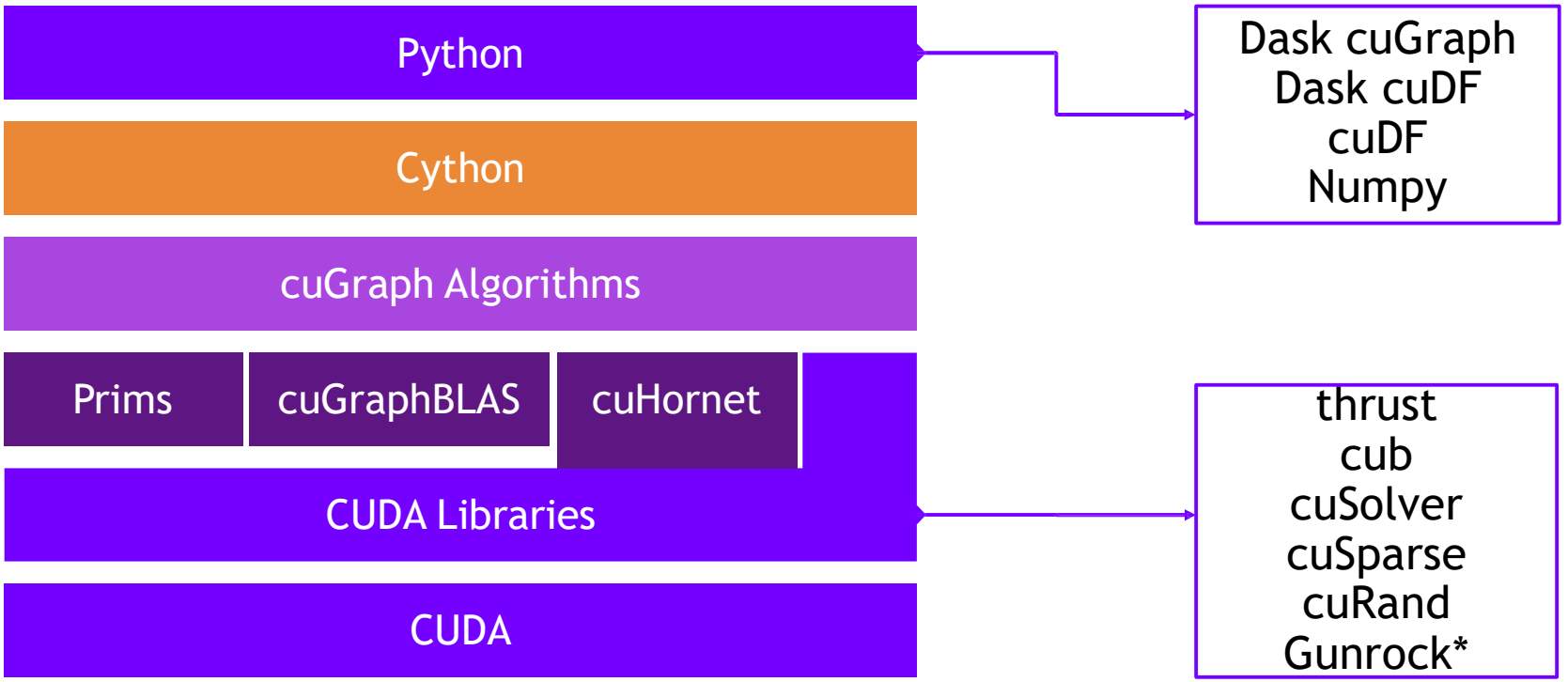
Seamless Integration with cuDF and cuML

- Property Graph support via DataFrames

Growing Functionality

- Extensive collection of algorithm, primitive, and utility functions

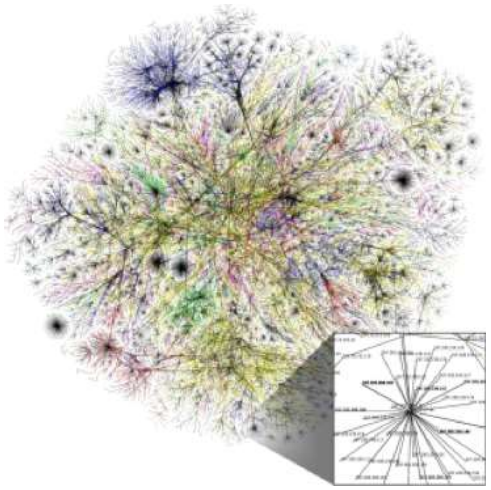
Graph Technology Stack



nvGRAPH has been Opened Sourced and integrated into cuGraph. A legacy version is available in a RAPIDS GitHub repo

* Gunrock is from UC Davis

Algorithms
GPU-accelerated NetworkX



Query Language

Multi-GPU

Utilities

More to come!

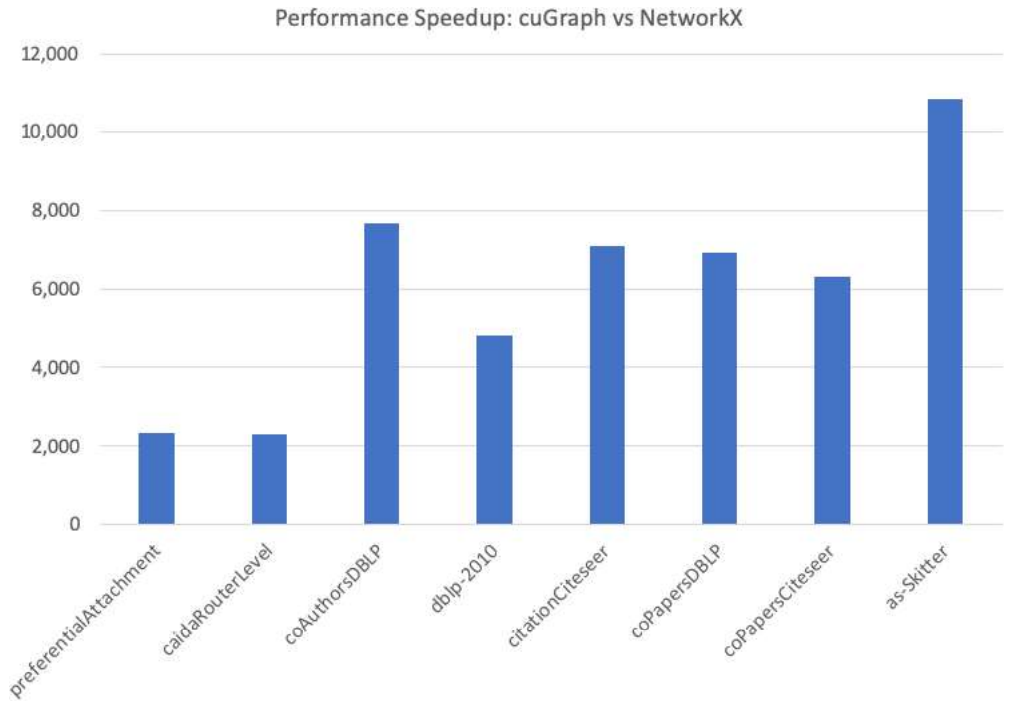
Community	Spectral Clustering Balanced-Cut Modularity Maximization Louvain Subgraph Extraction Triangle Counting
Components	Weakly Connected Components **Strongly Connected Components**
Link Analysis	Page Rank (**Multi-GPU**) Personal Page Rank
Link Prediction	Jaccard Weighted Jaccard Overlap Coefficient
Traversal	Single Source Shortest Path (SSSP) Breadth First Search (BFS)
Structure	COO-to-CSR (**Multi-GPU**) Transpose Renumbering

Louvain Single Run

```
G = cugraph.Graph()
G.add_edge_list(gdf["src_0"], gdf["dst_0"], gdf["data"])
df, mod = cugraph.nvLouvain(G)
```

Louvain returns:
cudf.DataFrame with two names columns:
louvain["vertex"]: The vertex id.
louvain["partition"]: The assigned partition.

Dataset	Nodes	Edges
preferentialAttachment	100,000	999,970
caidaRouterLevel	192,244	1,218,132
coAuthorsDBLP	299,067	299,067
dblp-2010	326,186	1,615,400
citationCiteseer	268,495	2,313,294
coPapersDBLP	540,486	30,491,458
coPapersCiteseer	434,102	32,073,440
as-Skitter	1,696,415	22,190,596



Road to 1.0
March 2020 - RAPIDS 0.14

cuGraph	Single-GPU	Multi-GPU	Multi-Node-Multi-GPU
Jaccard and Weighted Jaccard	✓	✓	✓
Page Rank	✓	✓	✓
Personal Page Rank	✓	✓	✓
SSSP	✓	✓	✓
BFS	✓	✓	✓
Triangle Counting	✓	✓	✓
Subgraph Extraction	✓	✓	✓
Katz Centrality	✓		
Betweenness Centrality	✓		
Connected Components (Weak and Strong)	✓	✓	✓
Louvain	✓		
Spectral Clustering	✓	✓	✓
InfoMap	✓		
K-Cores	✓		